ASE ’26 · MUNICH

Barista

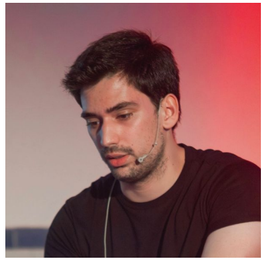
Synthesizing Typestate Specifications with LLM Agents



Catarina Gamboa



Paulo Canelas



Ricardo Costa



Márcio Caetano



Jonathan Aldrich



Alcides Fonseca

Carnegie Mellon University · LASIGE driven by excellence · Ciências ULisboa · S3D · Carnegie Mellon Portugal

Can you spot the bug?

```
1  Socket s =new Socket();
2  s.bind(addr);
3  s.getInputStream();
4
```

Classes carry implicit protocols that Java cannot check

```
1  Socket s =new Socket();
2  s.bind(addr);
3  s.getInputStream();
4
```

✅ Compiles

✅ Type-checks

⛔ `SocketException`

Only at runtime, and only on the paths the tests exercise.

Classes carry implicit protocols that Java cannot check

```
1  Socket s =new Socket();
2  s.bind(addr);
3  s.getInputStream();
4  // missing connect() before get
```

✅ Compiles

✅ Type-checks

⛔ `SocketException`

Only at runtime, and only on the paths the tests exercise.

Developers encode the protocol in the documentation

```
public InputStream getInputStream() throws IOException
```

IOException – if an I/O error occurs when creating the input stream, the socket is closed, **the socket is not connected**, …

javadoc of java.net.Socket

PROTOCOL MINING

Needs client code or execution traces

LLM SPEC GENERATION

Method-level pre- and postconditions only

BARISTA

Class-level protocols from documentation alone

Typestate specifications catch these violations at compile time

SocketRefinements.java

```
@StateSet({"unconnected", "bound",
           "connected", …})
@RefinementAlias(
  "ValidPort(int p) {0 <= p && p <= 65535}")

@StateRefinement(to="unconnected(this)")
void Socket();

@StateRefinement(from="unconnected(this)",
                 to="bound(this)")
void bind(SocketAddress b);

@StateRefinement(from="bound(this)",
                 to="connected(this)")
void connect(SocketAddress e);

@StateRefinement(from="connected(this)")
InputStream getInputStream();
```

THE IMPLICIT PROTOCOL, MADE EXPLICIT



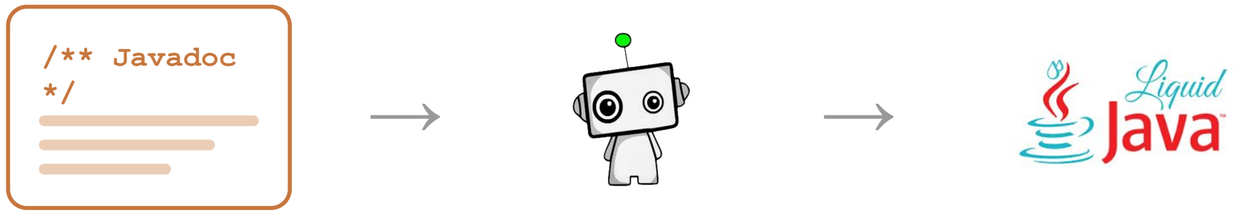

```
s = new Socket();
s.bind(addr);
s.getInputStream();
```

Compile error, line 3

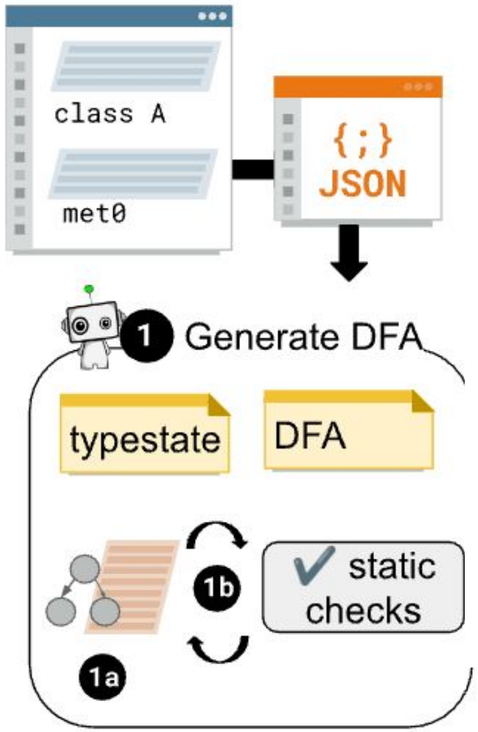
expected connected(s), found bound(s)

MAIN QUESTION

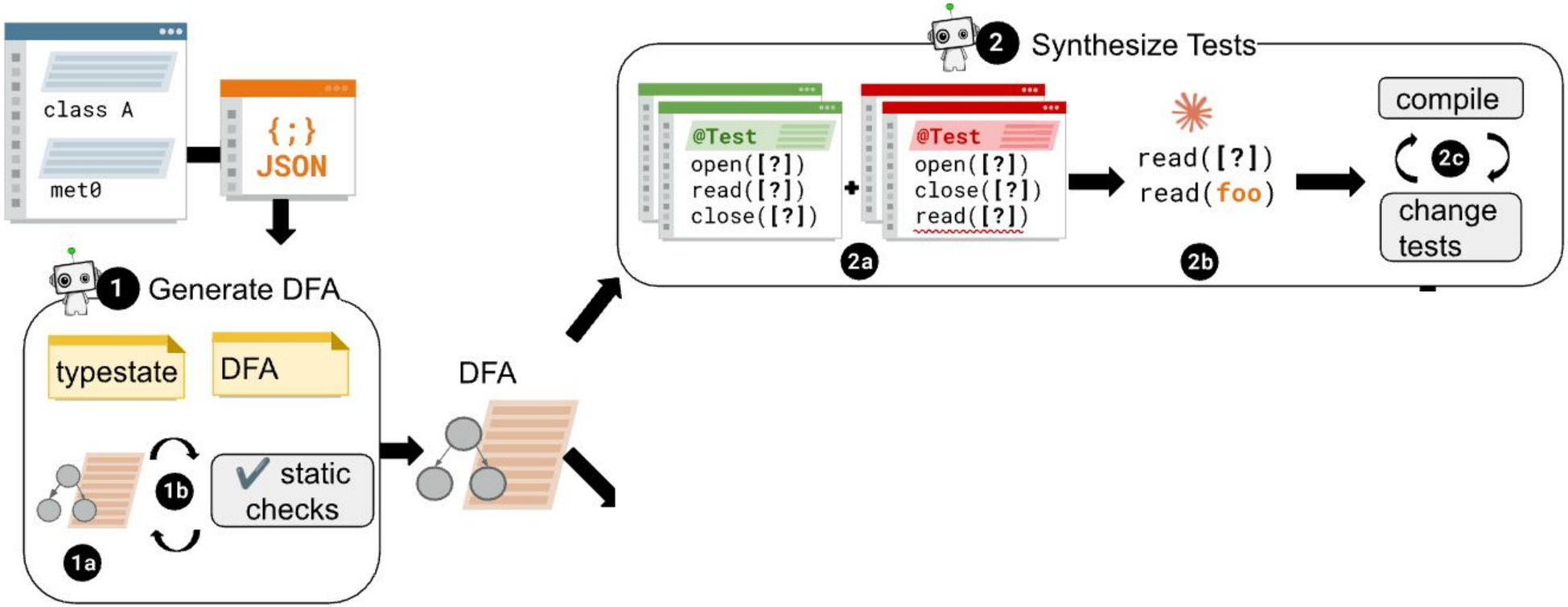
Can LLM agents turn the protocol described in documentation into a machine-verifiable specification?



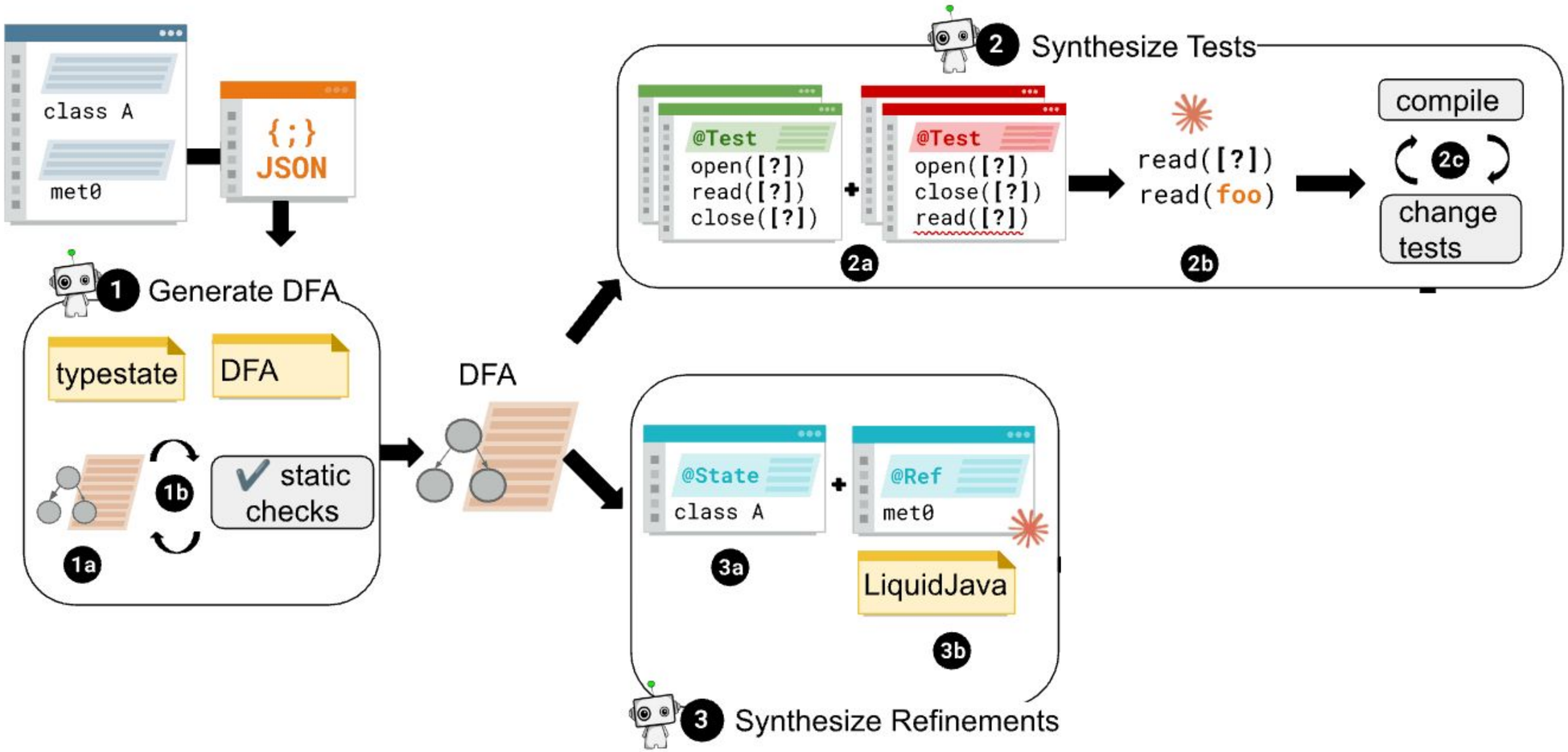
Barista turns documentation into verified LiquidJava specifications



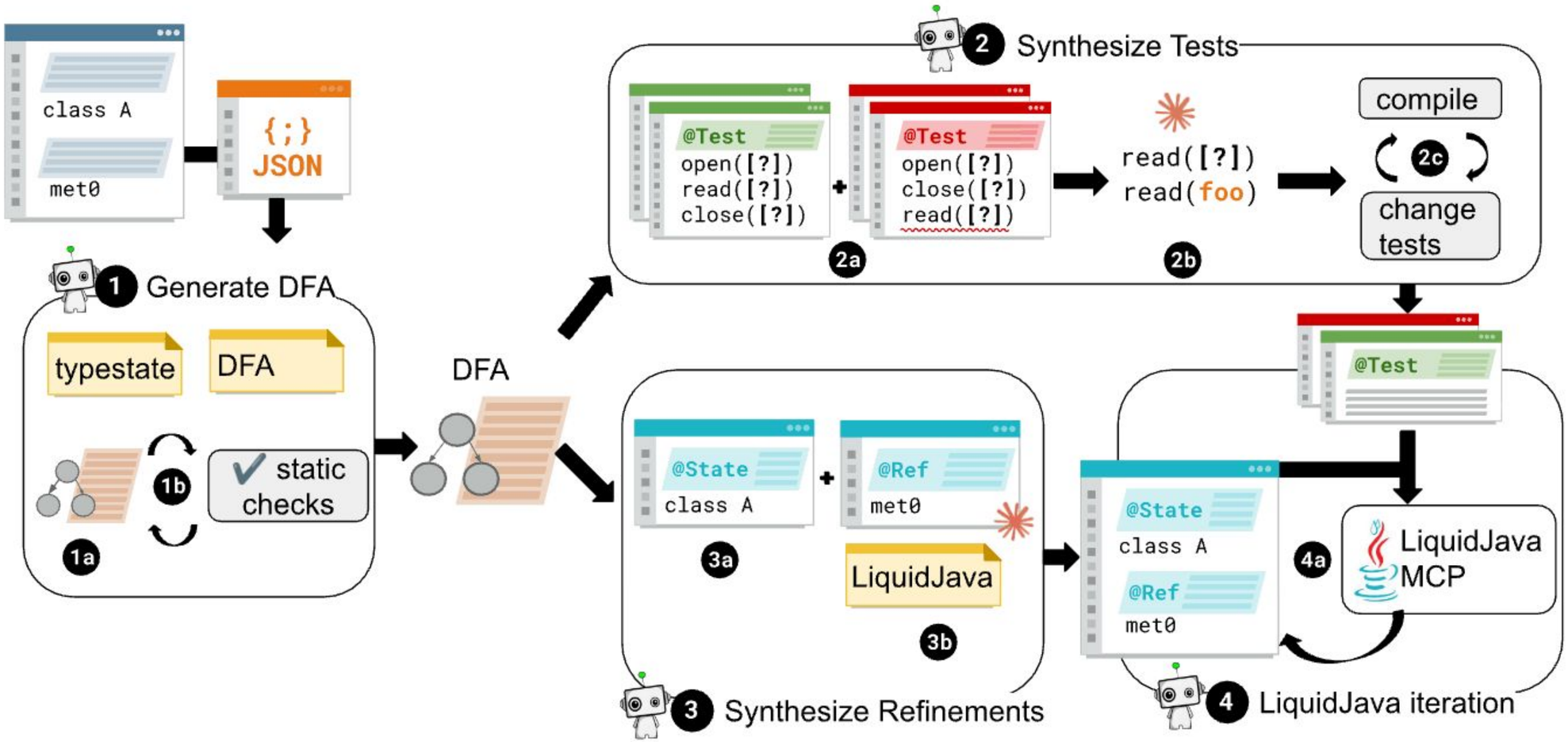
Barista turns documentation into verified LiquidJava specifications



Barista turns documentation into verified LiquidJava specifications

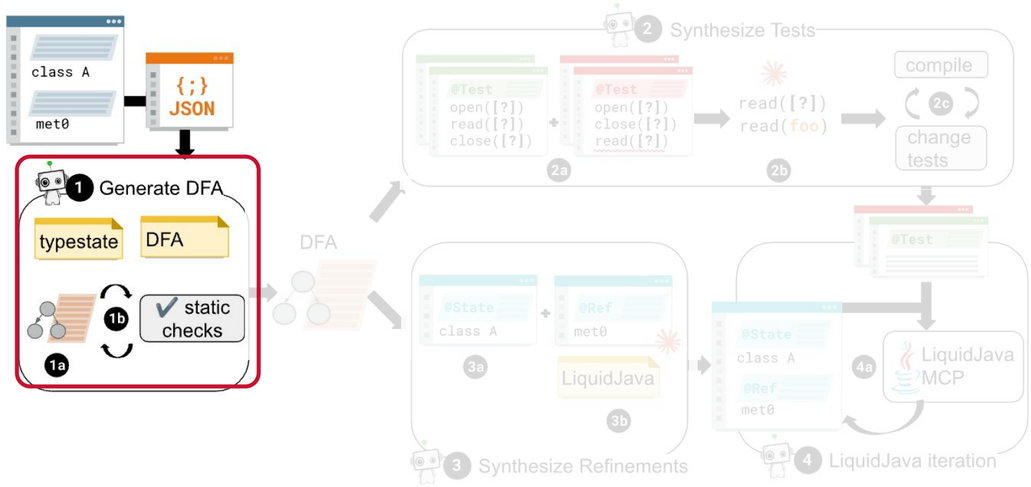


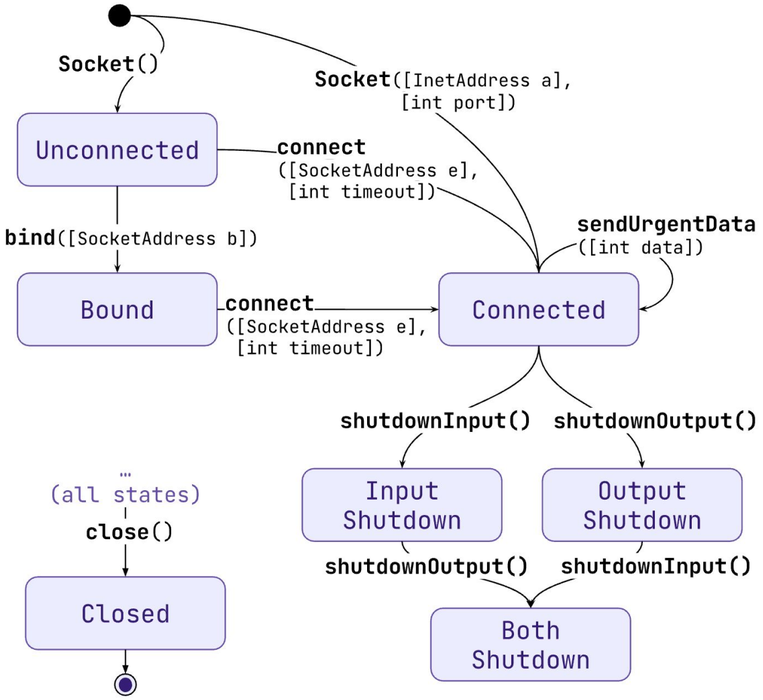
Barista turns documentation into verified LiquidJava specifications



BARISTA WORKFLOW · STEP 1

Step 1: the agent extracts the protocol as a DFA





in Javadoc as JSON

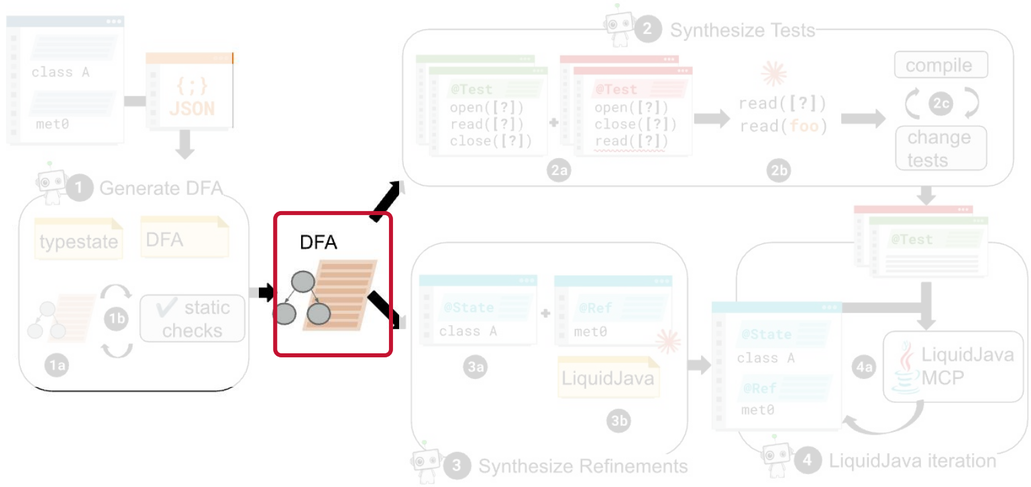
out Mermaid state machine

MCP VALIDATION, UNTIL ALL PASS

- Every public method appears
- Every state is reachable
- Transitions are deterministic

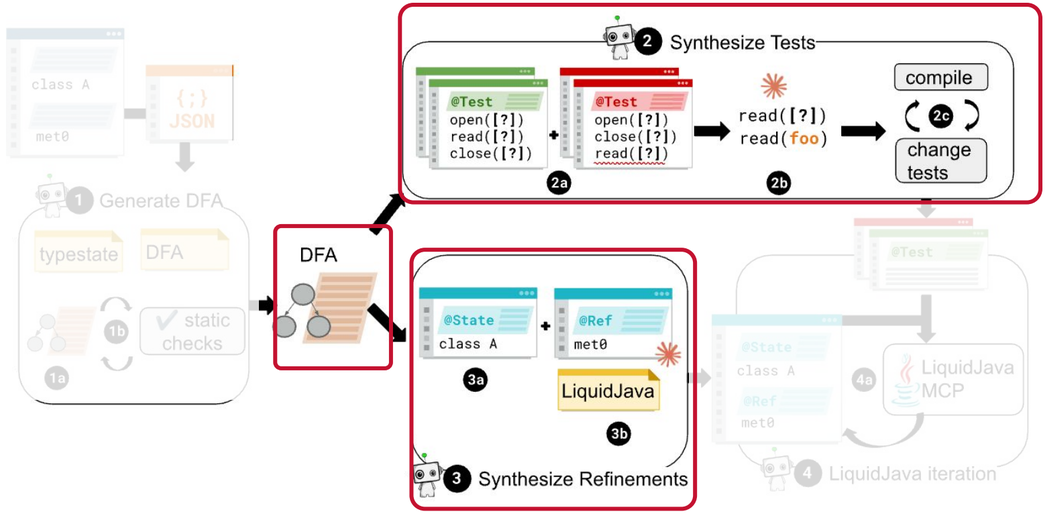
BARISTA WORKFLOW · STEPS 2–3

Steps 2–3: the DFA drives both tests and refinements



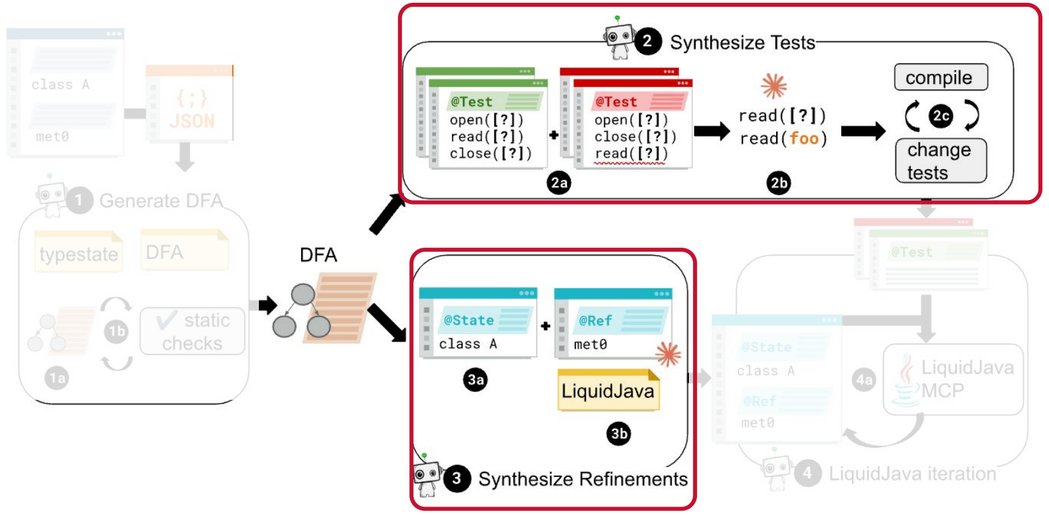
BARISTA WORKFLOW · STEPS 2–3

Steps 2–3: the DFA drives both tests and refinements



BARISTA WORKFLOW · STEPS 2–3

Steps 2–3: the DFA drives both tests and refinements



2 · SYNTHESIZE TESTS

DFA paths become test templates; the agent fills the `[?]` until they compile

@Test valid

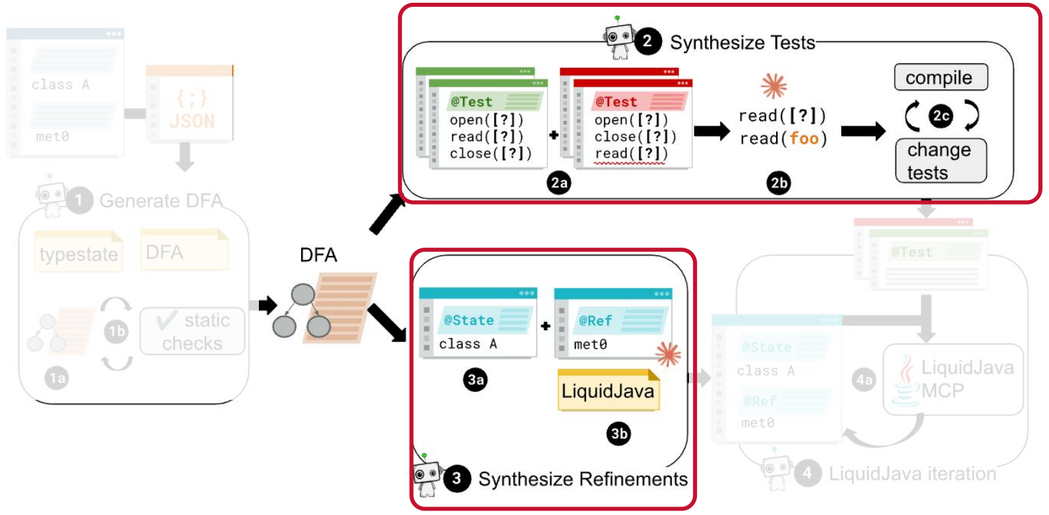
```
s = new Socket([InetAddress a], [int port]);
s.close();
```

@Test invalid

```
s = new Socket();
s.sendUrgentData([int data]);
```

BARISTA WORKFLOW · STEPS 2–3

Steps 2–3: the DFA drives both tests and refinements



2 · SYNTHESIZE TESTS

DFA paths become test templates; the agent fills the `[?]` until they compile

@Test valid

```
s = new Socket([InetAddress a], [int port]);
s.close();
```

@Test invalid

```
s = new Socket();
s.sendUrgentData([int data]);
```

3 · SYNTHESIZE REFINEMENTS

States and transitions map mechanically; the agent adds argument constraints

from DFA · MCP

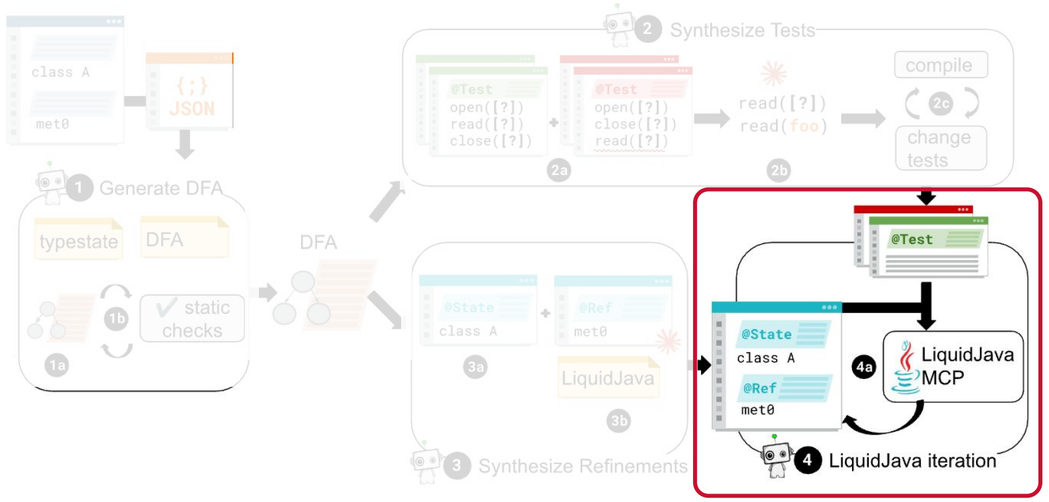
```
@StateRefinement(from="bound(this)",
                 to="connected(this)")
```

from docs · agent

```
Socket(InetAddress a,
  @Refinement("ValidPort(_)") int port)
```

BARISTA WORKFLOW · STEP 4

Step 4: LiquidJava checks the spec against the tests



`@State @Ref`

Annotated class

`@Test`+`@Test`

Valid and invalid tests

→

verifier via MCP

→

valid test passes

invalid test is rejected

Any mismatch returns the error message to the agent, which refines the spec until every test is classified correctly or it reaches max iterations

We evaluated on 35 JDK classes against expert ground truth

35

CLASSES

I/O, network, graphics, security, system, data

`Socket, ZipFile, BufferedReader, LoginContext, UndoManager`

2

EXPERTS PER CLASS

Independent, **no AI** , then reconciled

3,262

EXPERT TESTS

1,302 valid, **1,960** invalid

5

RUNS PER CLASS

To account for non-determinism

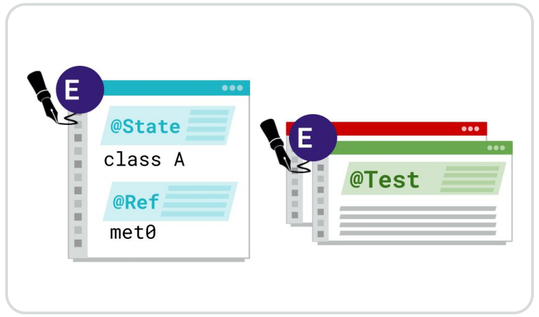
Classes from Beckman et al., ECOOP ’11

METHODOLOGY

The agent is never graded on its own work

GROUND TRUTH

Create specifications and tests covering DFA



Expert specs + expert tests

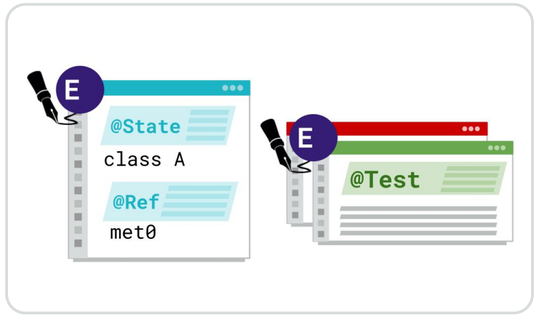
E expert agent

METHODOLOGY

The agent is never graded on its own work

GROUND TRUTH

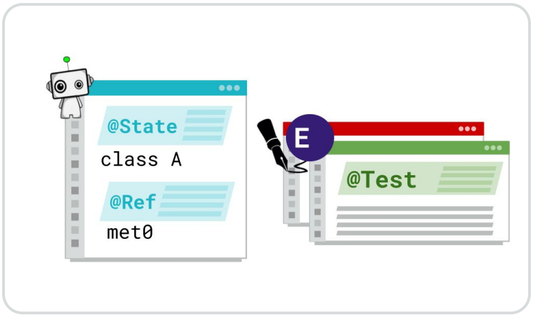
Create specifications and tests covering DFA



Expert specs + expert tests

RQ1

How effective are the generated specifications?



Agent specs vs expert tests

Precision, recall, F1, never on its own tests

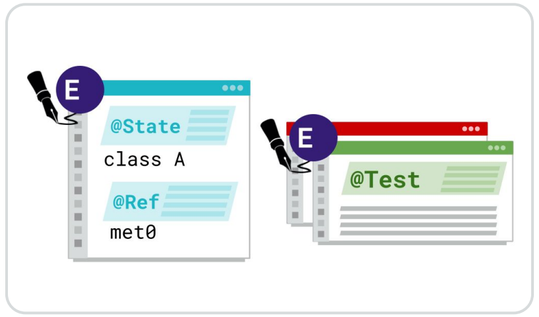
E expert agent

METHODOLOGY

The agent is never graded on its own work

GROUND TRUTH

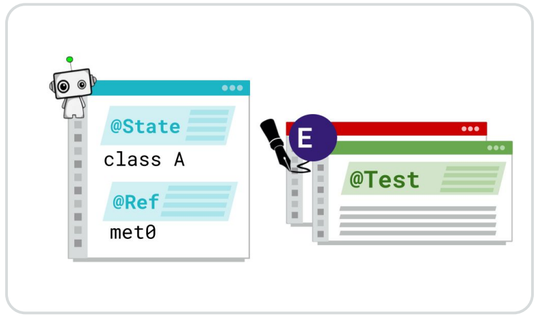
Create specifications and tests covering DFA



Expert specs + expert tests

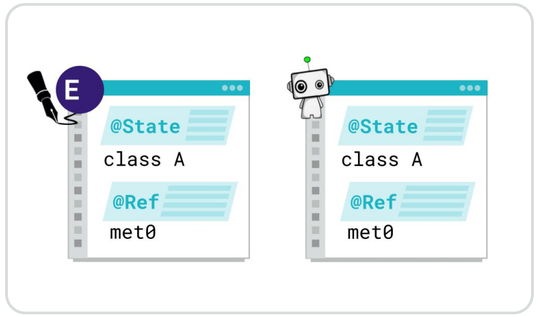
RQ1

How effective are the generated specifications?



Agent specs vs expert tests

Precision, recall, F1, never on its own tests



Agent specs vs expert specs

States, transitions, root causes of divergence

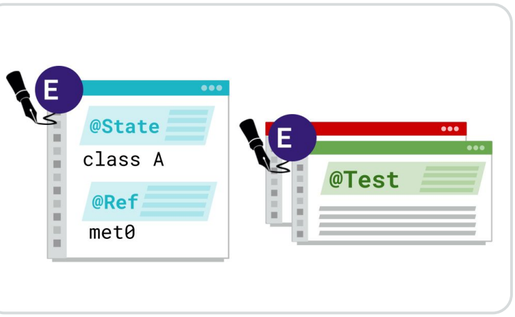
E expert agent

METHODOLOGY

The agent is never graded on its own work

GROUND TRUTH

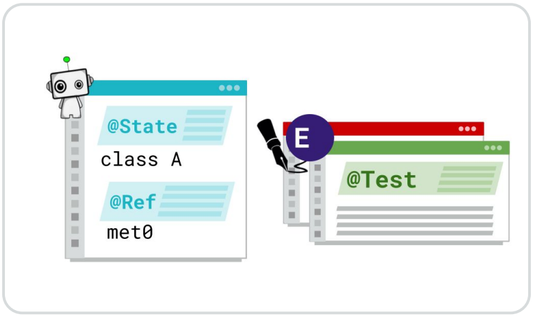
Create specifications and tests covering DFA



Expert specs + expert tests

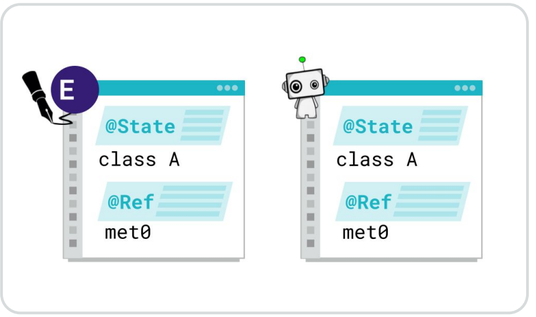
RQ1

How effective are the generated specifications?



Agent specs vs expert tests

Precision, recall, F1, never on its own tests

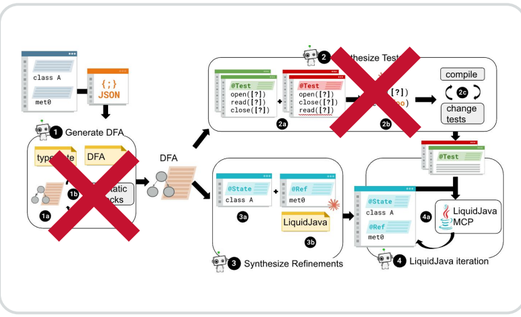


Agent specs vs expert specs

States, transitions, root causes of divergence

RQ2

Which design choices matter?



Ablations vs expert tests

Remove the DFA, the tests, the verifier and re-measure

E expert agent

RQ1.1 · DO SPECIFICATIONS SEPARATE VALID FROM INVALID USAGES?

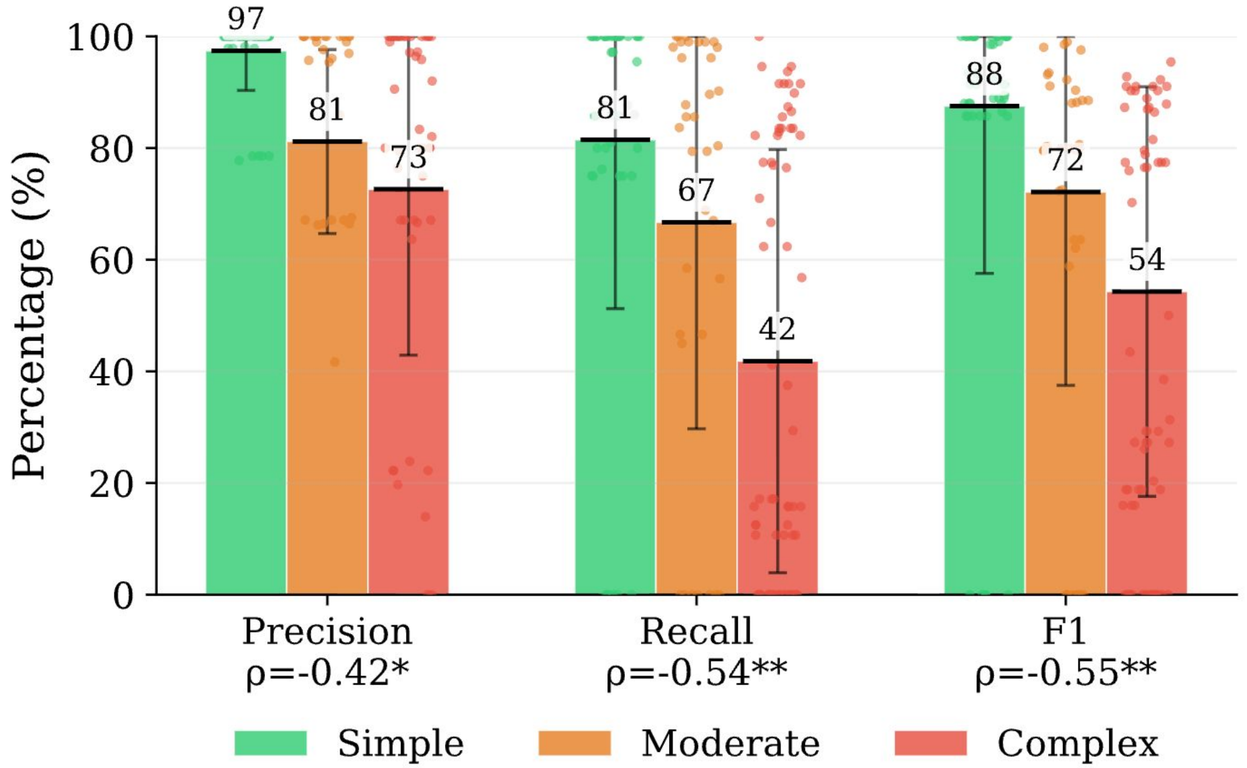
Barista reaches 87% precision, lower on complex protocols

87.0%

PRECISION
Flagged violations are real

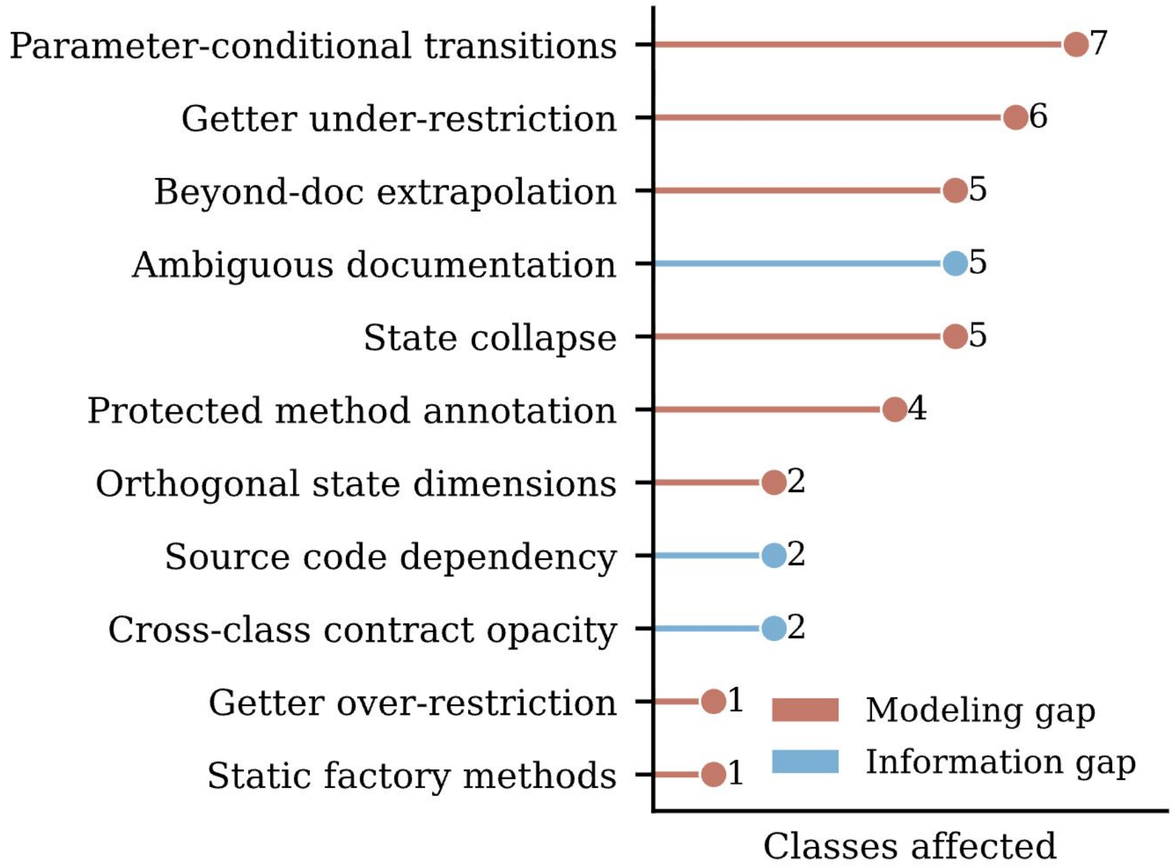
68.3%

RECALL
Of actual violations caught

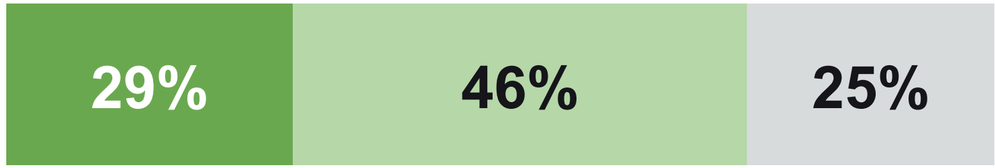


RQ1.2 · HOW DO SPECIFICATIONS ALIGN WITH THE EXPERTS'?

Most divergences are modeling gaps; 75% remain usable



USABILITY OF GENERATED SPECS



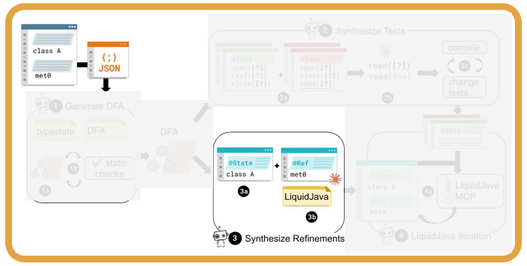
DIRECTLY USABLE
States and transitions match experts

MINOR EDITS
Core protocol right, local fixes needed

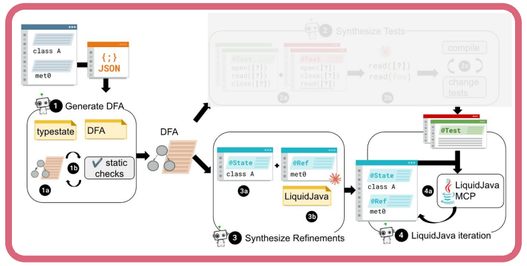
NOT USABLE
State machine needs redesign

Ablation: DFA raises precision, tests raise recall

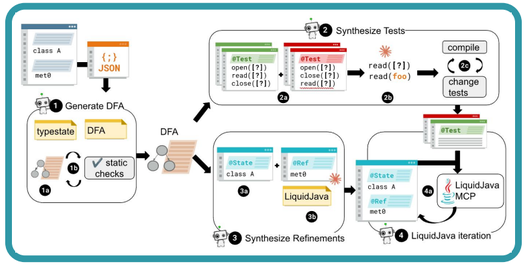
BASELINE docs → spec, few-shot

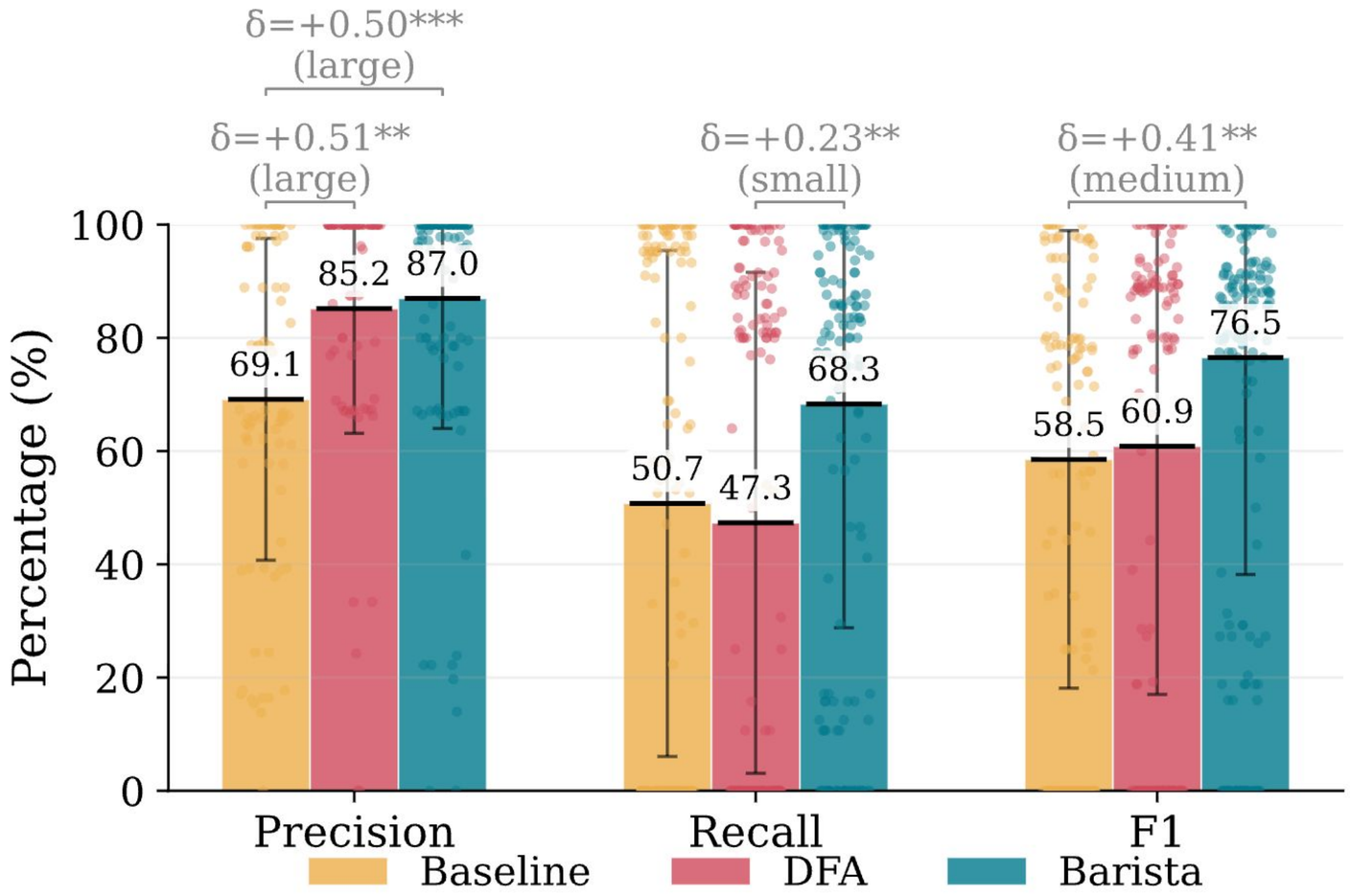


+ DFA no tests



BARISTA DFA drives tests too





Takeaways and next steps

1 **Documentation is enough to bootstrap typestate specs**

High precision; a starting point for experts, not a replacement

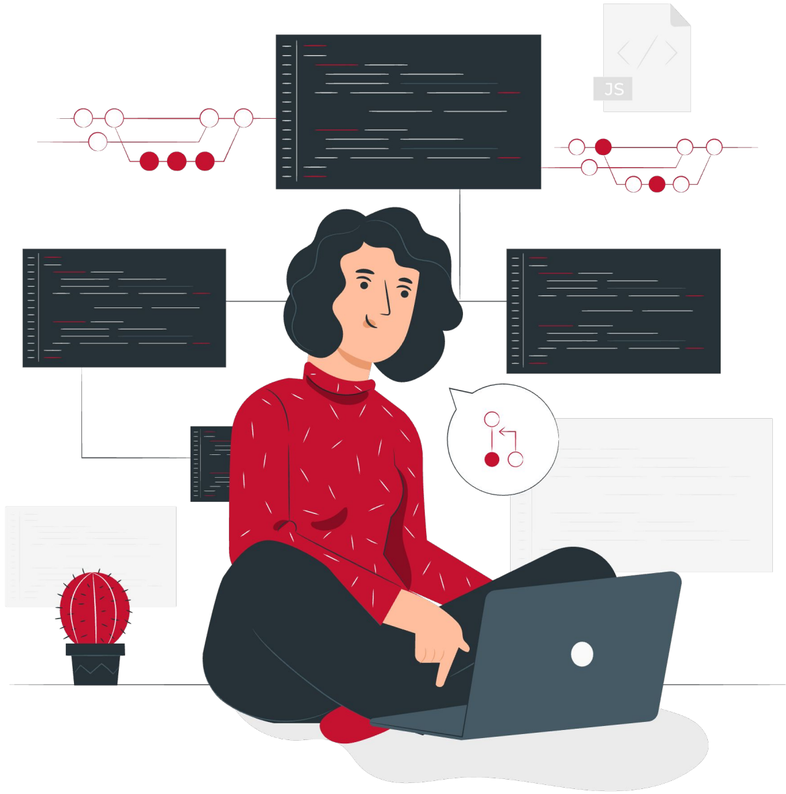


Illustration from storyset.com

Takeaways and next steps

1 **Documentation is enough to bootstrap typestate specs**

High precision; a starting point for experts, not a replacement

2 **Intermediate representation for precision, tests for recall**

The gains come from the pipeline, more than from multi-agent orchestration

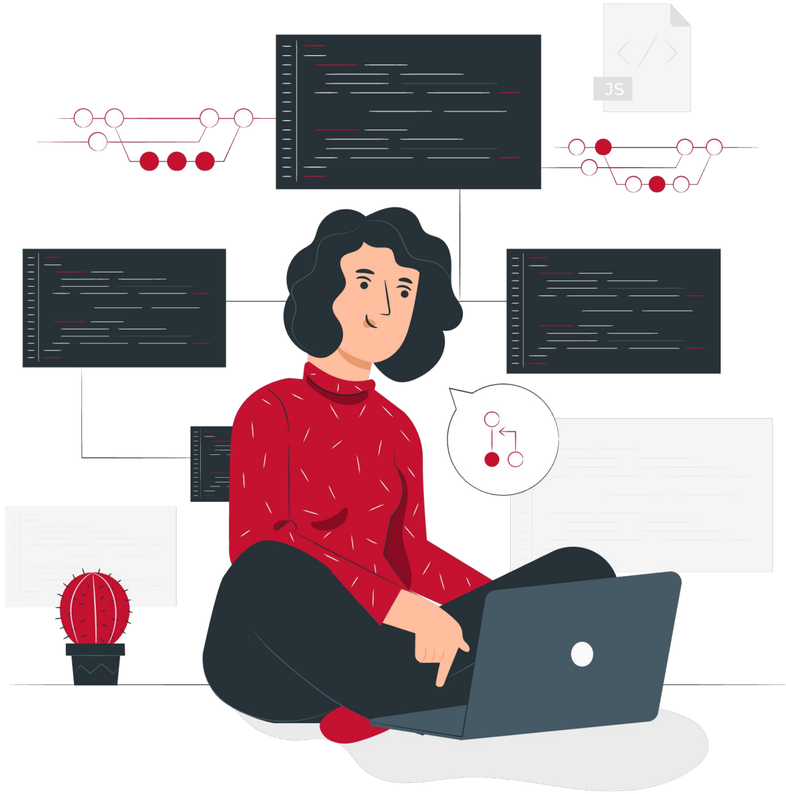


Illustration from storyset.com

Takeaways and next steps

1 **Documentation is enough to bootstrap typestate specs**

High precision; a starting point for experts, not a replacement

2 **Intermediate representation for precision, tests for recall**

The gains come from the pipeline, more than from multi-agent orchestration. They help developers understand object protocol.

3 **Next: richer representations than the DFA**

Symbolic automata or extended FSMs for parameter-conditional transitions

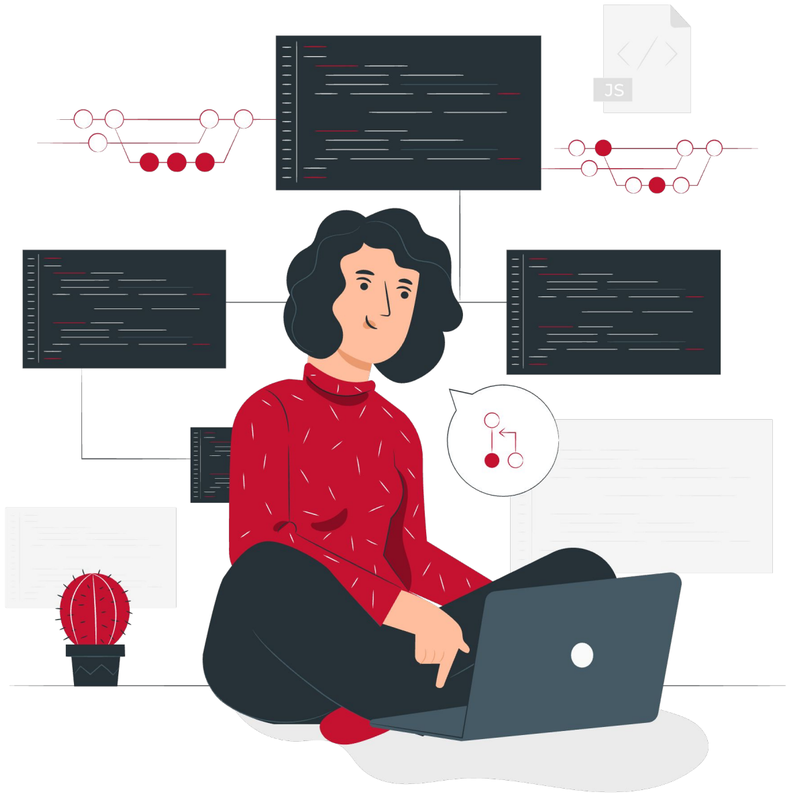
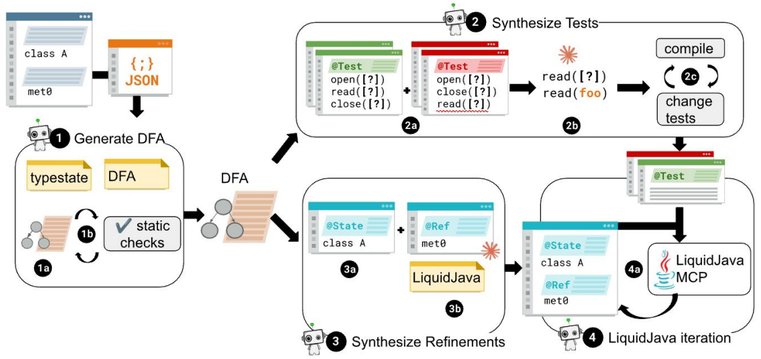


Illustration from storyset.com

Barista turns documentation into verified LiquidJava specifications

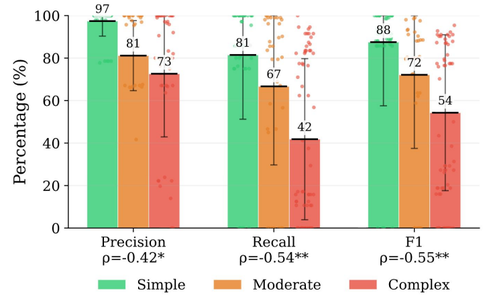


RQ1.1 · DO SPECIFICATIONS SEPARATE VALID FROM INVALID USAGES?

Barista reaches 87% precision, lower on complex protocols

87.0% **PRECISION**
Flagged violations are real

68.3% **RECALL**
Of actual violations caught



RQ2 · IMPACT OF DESIGN CHOICES

Ablation: DFA raises precision, tests raise recall

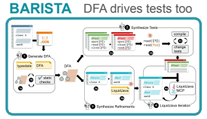
BASELINE docs → spec, few-shot

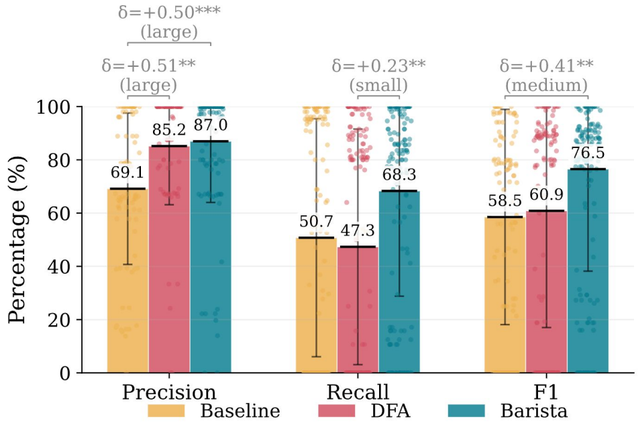


+ DFA no tests



BARISTA DFA drives tests too



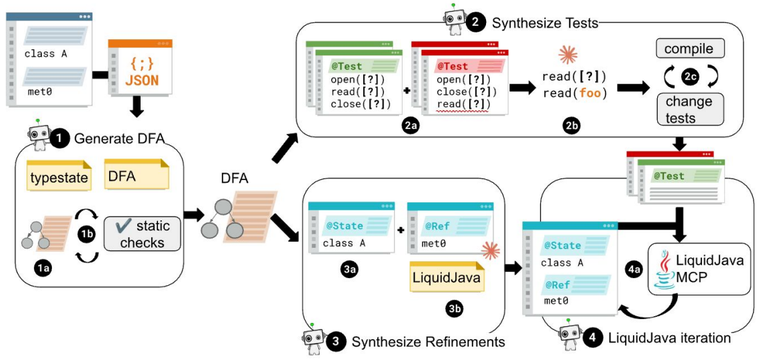


More details

Thank you

cgamboa@andrew.cmu.edu

Barista turns documentation into verified LiquidJava specifications

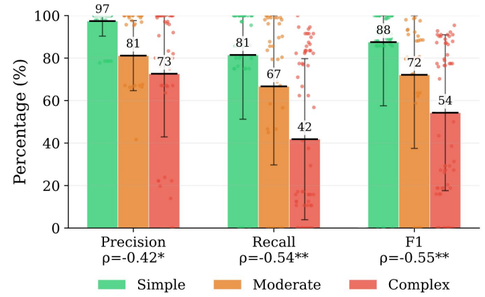


RQ1.1 · DO SPECIFICATIONS SEPARATE VALID FROM INVALID USAGES?

Barista reaches 87% precision, lower on complex protocols

87.0% **PRECISION**
Flagged violations are real

68.3% **RECALL**
Of actual violations caught



RQ2 · IMPACT OF DESIGN CHOICES

Ablation: DFA raises precision, tests raise recall

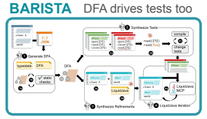
BASELINE docs → spec, few-shot

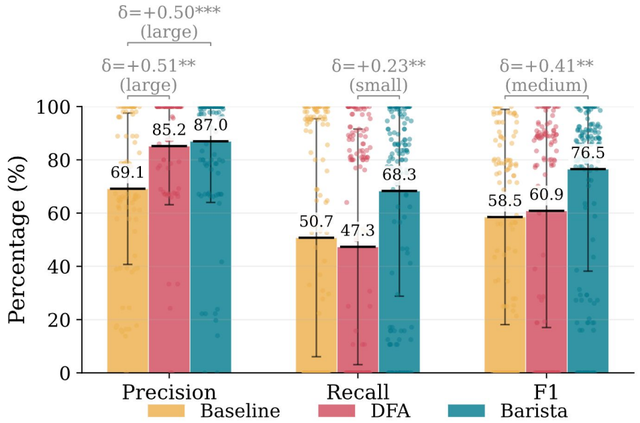


+ DFA no tests



BARISTA DFA drives tests too





More details

Thank you

cgamboa@andrew.cmu.edu

Table 2: Confusion matrix used to label each test case in the cross-evaluation protocol.

	LiquidJava Error	LiquidJava Passes
Invalid path	True Positive (TP) *correct detection*	False Negative (FN) *under-restriction*
Valid path	False Positive (FP) *over-restriction*	True Negative (TN) *correct acceptance*

Table 3: Performance metrics for Barista.

	Accuracy	Precision	Recall	F1
Barista	74.8%	87.0%	68.3%	76.5%

$$\textbf{Precision} = \frac{\text{True Positive}}{\text{Actual Results}} \quad \text{or} \quad \frac{\text{True Positive}}{\text{True Positive + False Positive}}$$

$$\textbf{Recall} = \frac{\text{True Positive}}{\text{Predicted Results}} \quad \text{or} \quad \frac{\text{True Positive}}{\text{True Positive + False Negative}}$$

$$\textbf{Accuracy} = \frac{\text{True Positive + True Negative}}{\text{Total}}$$

